BDH Architecture Analysis: A Controlled Component-Level Study of the Dragon Hatchling Language Model

Aditya Khamitkar*, Tushar Jagatap*, Nitin Saini*
*Under the Guidance of Dr. Lakshmi D.
SCAI, VIT Bhopal University
{adityakhamitkar2022, tusharjagannathjagatap2022, nitinsaini2022}@example.edu

***Abstract*—The Dragon Hatchling (BDH) is a biologically-inspired large language model architecture introduced by Kosowski et al. (arXiv:2509.26507) that combines graph-theoretic foundations, Hebbian synaptic plasticity, sparse positive activations, and linear attention into a single state-space framework. While the original paper demonstrates that BDH rivals GPT-2 at parameter parity (10 M–1 B), the relative contribution of each design decision remains under-studied.**

This paper presents a *controlled, component-level ablation study* of BDH on byte-level WikiText-2 language modelling. We isolate and evaluate four design dimensions: (1) multiplicative interaction between primal and dual latent branches, (2) latent dimensionality governed by the internal multiplier, (3) sparse activation function (ReLU vs. GELU), and (4) the core attention mechanism. Each experiment modifies exactly one component, holding all others constant, so that performance differences are causally attributable.

Our experimental results across 2,900 training steps reveal that *multiplicative interaction is the most critical component* (removing it raises loss by $+0.059$), while switching from ReLU to GELU yields modest but consistent improvement (-0.039), and reducing latent dimensionality incurs only a marginal penalty (-0.052 relative to base), suggesting that the full-dimensional BDH may be over-parameterised at the small scale tested. We provide detailed architecture diagrams, full PyTorch code listings, and analysis of training dynamics, contributing an open, reproducible reference implementation for future BDH research.

***Index Terms*—Dragon Hatchling, BDH, biologically-inspired neural networks, ablation study, multiplicative interaction, sparse activation, linear attention, language modelling, WikiText-2, PyTorch.**

1. Introduction

1.1. Background and Motivation

The relationship between artificial neural networks and biological intelligence has motivated researchers since the seminal work of McCulloch and Pitts [7] and the formalisation of the Turing Machine [8]. The Transformer architecture [2], which now underpins virtually all state-of-the-art large language models, achieves remarkable performance but diverges significantly from biologically realistic computation: it relies on dense, synchronous, globally-coupled attention; it lacks sparse, positive activation states; and its working-memory mechanism (the KV cache) bears no resemblance to synaptic plasticity.

Kosowski et al. (2025) [1] introduce the **Baby Dragon Hatchling (BDH)**, a fundamentally different architecture grounded in four key principles simultaneously:

- **P1. Scale-free graph dynamics.** BDH is formally a local, distributed computation on a graph of n neuron particles. The resulting network exhibits high modularity and a heavy-tailed degree distribution—hallmarks of biological neural connectivity [6].
- **P2. Hebbian synaptic plasticity.** Working memory during inference is maintained entirely through edge-reweighting governed by Hebbian learning rules, mimicking potentiation observed in biological neurons over minute-scale timescales.
- **P3. Sparse, positive activations and monosemanticity.** Activation vectors are constrained to be non-negative and sparse, enabling single-concept (monosemantic) neuron representations that emerge naturally without superposition.
- **P4. GPU-friendly tensor formulation.** Despite its graph theoretic description, BDH admits a compact state-space representation (BDH-GPU) that maps onto standard matrix multiplication, making it competitive with the Transformer in wall-clock training speed.

While the original paper establishes theoretical foundations and reports scaling results comparable to GPT-2, it does not systematically identify which individual components account for BDH's performance. This is the gap we address.

1.2. Research Questions

This study answers four tightly focused questions:

- **RQ1. Multiplicative interaction.** The core BDH novelty is element-wise multiplication of the primal latent

vector $\mathbf{x}_s$ and the attention-modulated dual vector $\mathbf{y}_s$. How much does replacing this product with addition degrade performance?

RQ2. Latent dimensionality. The BDH internal state is controlled by a multiplier m that scales $N = d \cdot m/H$ (dimension per head). What happens if m is reduced from 128 to 32?

RQ3. Activation function. BDH's biological plausibility depends on the sparse, non-negative ReLU. Does switching to the smoother GELU improve predictive loss at a cost to interpretability?

RQ4. Attention mechanism contribution. How essential is the causal self-attention inside each BDH layer, and how does the full BDH stack compare to a vanilla Transformer baseline of identical parameter budget?

1.3. Contributions

C1. We provide the first systematic, controlled ablation of BDH components on a standard language modelling benchmark.

C2. We implement and open-source five model variants—Transformer baseline, BDH Base, BDH-NoMul, BDH-LowDim, and BDH-Improved—in clean PyTorch with a fully reproducible training pipeline.

C3. We contribute a suite of TikZ architecture diagrams, training curves, and component-impact visualisations for use by the community.

C4. We derive observations about the relative importance of BDH design choices that can guide future architecture search.

2. Related Work

2.1. Transformer and Its Variants

The Transformer [2] replaces recurrence with dot-product self-attention, achieving $O(T^2 d)$ time complexity per layer. GPT-2 [3] demonstrates that autoregressive Transformers can model language across scales from 117 M to 1.5 B parameters, establishing the scaling benchmarks that BDH targets. Numerous efficient variants address the $O(T^2)$ attention bottleneck: Linformer [11] uses random projections; Performer [12] approximates softmax attention via FAVOR+; and RetNet [13] introduces a recurrent equivalent. BDH is most closely related to *linear attention* models, differing in its biological motivation and multiplicative latent interaction.

2.2. State-Space Models

Gu et al. [4] introduce the Structured State Space (S4) model, followed by Mamba [5] which adds input-selective state transitions. These SSMs achieve Transformer-comparable perplexity while maintaining $O(T)$ inference time. BDH-GPU can be expressed as a state-space system, placing it in this family, but its derivation from graph dynamics and Hebbian plasticity is conceptually distinct.

2.3. Biologically Plausible Networks

Spiking Neural Networks (SNNs) [14] and integrate-and-fire models [15] predate deep learning but rarely match Transformer-scale language performance. BDH bridges this gap by showing that integrate-and-fire thresholding corresponds exactly to ReLU sparsification, and that Hebbian edge-reweighting is equivalent to a form of linear attention.

2.4. Interpretability and Monosemanticity

Elhage et al. [9] demonstrate that Transformer neurons are often *polysemantic*—representing multiple unrelated concepts in superposition. Sparse autoencoders [10] are subsequently used to recover monosemantic features. BDH achieves monosemanticity by construction through its sparse, non-negative activations, without requiring post-hoc disentanglement.

3. The BDH Architecture

This section reviews the theoretical foundations of BDH at the level of detail required to understand our ablations. We follow the notation of Kosowski et al. [1].

3.1. High-Level Overview

Figure 1 shows the complete BDH forward pass. The model processes a token sequence of length T with batch size B. Let d denote the embedding dimension ($d = 256$ in our experiments), H the number of heads ($H = 4$), m the internal multiplier ($m = 128$ for BDH Base), and $N = dm/H$ the per-head latent dimension.

3.2. Formal Definition of BDH-GPU

Given input $\mathbf{x} \in \mathbb{R}^{B\times 1\times T\times d}$ (after embedding and normalisation), each layer computes:

$$\mathbf{x}_\ell = \mathbf{x}\,\mathbf{W}_{\text{enc}} \qquad \in \mathbb{R}^{B\times H\times T\times N} \tag{1}$$

$$\mathbf{x}_s = \mathrm{ReLU}(\mathbf{x}_\ell) \qquad \text{(sparse primal)} \tag{2}$$

$$\mathbf{y} = \mathrm{Attn}(\mathbf{x}_s, \mathbf{x}_s, \mathbf{x}) \qquad \text{(causal attention)} \tag{3}$$

$$\mathbf{y} = \mathrm{LN}(\mathbf{y}) \tag{4}$$

$$\mathbf{y}_\ell = \mathbf{y}\,\mathbf{W}_{\text{enc\_v}} \tag{5}$$

$$\mathbf{y}_s = \mathrm{ReLU}(\mathbf{y}_\ell) \qquad \text{(sparse dual)} \tag{6}$$

$$\mathbf{z} = \mathbf{x}_s \odot \mathbf{y}_s \qquad \textbf{multiplicative gate} \tag{7}$$

$$\mathbf{y}_{\text{mlp}} = \mathrm{reshape}(\mathbf{z})\,\mathbf{W}_{\text{dec}} \tag{8}$$

$$\mathbf{x} \leftarrow \mathrm{LN}\big(\mathbf{x} + \mathrm{LN}(\mathbf{y}_{\text{mlp}})\big) \tag{9}$$

The attention in Equation (3) is causal self-attention:

$$\mathrm{Attn}(\mathbf{Q},\mathbf{K},\mathbf{V}) = \mathrm{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{D}} + \mathbf{M}\right)\mathbf{V} \tag{10}$$

where $D = N$ is the per-head dimension and $\mathbf{M}$ is the $T \times T$ causal mask ($-\infty$ for future positions).

3.3. Parameter Count

The learnable parameters are:

- Embedding matrix $\mathbf{E} \in \mathbb{R}^{V \times d}$
- Encoder $\mathbf{W}_{\text{enc}} \in \mathbb{R}^{H \times d \times N}$
- Encoder-V $\mathbf{W}_{\text{enc\_v}} \in \mathbb{R}^{H \times d \times N}$
- Decoder $\mathbf{W}_{\text{dec}} \in \mathbb{R}^{HN \times d}$
- Layer norm scale and bias (2 per LN, multiple LNs)
- LM head $\in \mathbb{R}^{d \times V}$

No positional encoding is used—temporal ordering is captured implicitly through the causal attention mask.

3.4. Biological Interpretation

As shown in Figure 2, BDH maps directly onto a two-circuit spiking neural architecture. The ReLU function implements integrate-and-fire thresholding: a neuron fires (positive activation) only when its total synaptic input exceeds its resting threshold of zero. The element-wise product in Equation (7) captures *conjunctive binding*—a neuron in the product space fires only when both the excitatory and inhibitory circuits are active, encoding compound concepts with high specificity (monosemanticity).

Crucially, the attention state matrix ($\mathbf{K}^\top \mathbf{V}$ in linear attention form) plays the role of Hebbian synaptic weights: after processing token t, the matrix accumulates $\mathbf{k}_t \mathbf{v}_t^\top$, increasing the connection strength between neurons that co-activate.

4. Experimental Variants

Figure 3 presents the five architectures side-by-side. We describe each variant below, then summarise design differences in Table 1.

4.0.1. Transformer Baseline. A standard decoder-only Transformer with causal multi-head attention, pre-norm LayerNorm, GELU feed-forward network (FFN dimension $4d = 1024$), and residual connections. QKV projections are fused into a single linear layer.

4.0.2. BDH Base. The full BDH-GPU architecture as described in Section 3. Uses $m = 128$, ReLU activations, and element-wise multiplication.

4.0.3. BDH-NoMul. Identical to BDH Base except the multiplicative gate $\mathbf{x}_s \odot \mathbf{y}_s$ is replaced by addition $\mathbf{x}_s + \mathbf{y}_s$. This tests the hypothesis that conjunction rather than superposition is necessary for BDH's inductive bias.

4.0.4. BDH-LowDim. The internal multiplier is reduced from $m = 128$ to $m = 32$, shrinking the per-head latent dimension N by a factor of 4. All other components are identical to BDH Base.

TABLE 1. ARCHITECTURAL DIFFERENCES BETWEEN VARIANTS

Model	Mult.	Activation	m	Arch
Transformer	—	GELU	—	MHA+FFN
BDH Base	$\odot$	ReLU	128	BDH-GPU
BDH-NoMul	$+$	ReLU	128	BDH-GPU
BDH-LowDim	$\odot$	ReLU	32	BDH-GPU
BDH-Improved	$\odot$	GELU	128	BDH-GPU

TABLE 2. HYPERPARAMETER CONFIGURATION

Hyperparameter	Value
Vocabulary size (V)	256
Layers (L)	6
Embedding dimension (d)	256
Attention heads (H)	4
Block size (T)	128
Batch size (B)	8
BDH multiplier (m, Base)	128
BDH multiplier (m, LowDim)	32
FFN dimension (Transformer)	1024
Dropout (p)	0.1
Optimiser	AdamW
Learning rate	1×10^{-3}
Weight decay	0.1
Training steps	2,900
Log / eval frequency	every 100 steps
Validation eval iterations	20
Random seed	42

4.0.5. BDH-Improved. Identical to BDH Base except ReLU is replaced by GELU in both the primal and dual sparse branches. GELU is a smoother approximation of ReLU: $\text{GELU}(x) = x\,\Phi(x)$ where Φ is the Gaussian CDF. This test evaluates whether the exact sparsity pattern enforced by ReLU is required, or whether a softer non-linearity suffices.

5. Experimental Setup

5.1. Dataset: WikiText-2 (Byte-Level)

We use WikiText-2 [17], a standard language modelling benchmark consisting of curated English Wikipedia articles. Rather than word-level or sub-word tokenisation, we encode the corpus at the *raw byte level*: every UTF-8 byte is a token, yielding a vocabulary of $V = 256$. This choice is biologically motivated—BDH is designed to process raw sensory sequences—and avoids confounds from tokeniser design.

The dataset is preprocessed by concatenating train, validation, and test splits into a single byte stream, removing empty lines, and saving as a memory-mapped binary file. A 90/10 train/validation split is applied on byte count.

TABLE 3. RESULTS SUMMARY — WIKITEXT-2, BYTE-LEVEL, 2,900 STEPS

Model	Init Loss	Final Loss	Avg Last-50	Std Last-50	Δ vs. Base
Transformer	5.777	**2.044**	**2.372**	0.658	−0.182
BDH Base	5.717	2.363	2.554	0.592	0.000
BDH-Improved	5.687	2.228	2.515	0.598	−0.039
BDH-LowDim	5.675	2.147	2.502	0.603	−0.052
BDH-NoMul	5.634	2.485	2.613	0.568	+0.059

$\Delta > 0$: degradation vs. Base; $\Delta < 0$: improvement vs. Base.

5.2. Training Pipeline

The training pipeline (Figure 4) is implemented in `train/train.py`. At each step, a random batch of $B = 8$ non-overlapping windows of length $T = 128$ is sampled from the training data. Validation loss is estimated every 100 steps using 20 random validation batches. Training runs for 2,900 steps (30 log points per model).

5.3. Evaluation Metric

The primary metric is **cross-entropy loss** on byte-level sequences. We report:

1) **Final loss**: training loss at step 2,900.
2) **Average last-50 loss**: mean of the last 50 logged training losses (i.e. steps 2,400–2,900 at 100-step frequency, so the last 5 log points plus earlier ones).
3) **Δ component impact**: difference in average last-50 loss between a BDH variant and BDH Base; positive values indicate degradation.

Note that the validation loss estimated by `estimate_loss` is used for monitoring; the logged loss is the training batch loss.

6. Results

6.1. Quantitative Summary

Table 3 presents the main results. Several observations stand out:

Observation 1 (Transformer leads at this scale)**.** *The Transformer achieves the best average last-50 loss of 2.372, roughly 0.18 nats lower than BDH Base. At the scale of 2,900 steps and $d = 256$, the Transformer's dense feed-forward block with $4\times$ expansion provides more representational capacity than the BDH's shared encoder/decoder parameter structure.*

Observation 2 (Multiplicative interaction is critical)**.** *BDH-NoMul (additive interaction) achieves an average last-50 loss of 2.613, which is $+0.059$ higher than BDH Base. This represents the largest single-component degradation, confirming that element-wise multiplication is not merely decorative but a functionally necessary operation.*

Observation 3 (Reducing latent dimension has minimal cost)**.** *BDH-LowDim ($m = 32$) achieves 2.502, slightly* better *than BDH Base (2.554), despite using only $1/4$ of the latent parameters per head. This suggests that BDH Base is over-parameterised in the latent space at this scale—a finding with significant implications for parameter efficiency.*

Observation 4 (GELU modestly outperforms ReLU)**.** *BDH-Improved (GELU) achieves 2.515 vs. BDH Base's 2.554, a -0.039 improvement. While GELU sacrifices the strict biological analogy of integrate-and-fire, it provides a smoother gradient landscape that benefits training.*

6.2. Training Dynamics

Figure 5 illustrates the training dynamics. All models start near ≈ 5.7 nats (close to the entropy of a random byte, $\log 256 \approx 5.55$ nats) and decrease steadily. Notably:

- The Transformer exhibits the steepest initial descent in the first 500 steps, consistent with its more expressive GELU FFN.
- BDH-LowDim and BDH-Improved show accelerated convergence in the second half of training (steps 1500+), ultimately surpassing BDH Base in final loss despite equal or fewer parameters.
- BDH-NoMul converges more slowly and plateaus at a higher loss, indicating that addition provides a qualitatively weaker learning signal than multiplication.

6.3. Component Impact Analysis

Figure 6 visualises the signed Δ for each component. The asymmetry is striking: the one change that *hurts* (removing multiplication) has a larger absolute magnitude than either of the two changes that *help* (GELU, lower dim). This supports the view that BDH's multiplicative gate is the single most critical architectural choice, while the hyperparameter choices for activation and latent dimension warrant further tuning at this scale.

7. Code Analysis

This section presents and annotates the key PyTorch implementations.

7.1. BDH Base Forward Pass

```
def forward(self, idx, targets=None):
    B, T = idx.shape

    # 1. Token embedding + head broadcast
    x = self.embed(idx).unsqueeze(1)    # (B, 1, T, d)
    x = self.ln(x)                      # LayerNorm

    for _ in range(self.n_layer):

        # ---- PRIMAL BRANCH ----
        # Project to latent space: (B, H, T, N)
        x_latent = x @ self.encoder      # matrix multiply
        x_sparse = F.relu(x_latent)     # integrate-and-
             fire
```

```
        # ---- ATTENTION (Q=K=sparse primal, V=full x) ----
        y = self.attn(x_sparse, x_sparse, x)  # (B,H,T,N)
        y = self.ln(y)

        # ---- DUAL BRANCH ----
        # Project attention output to latent space
        y_latent = y @ self.encoder_v
        y_sparse = F.relu(y_latent)    # sparse dual

        # ---- MULTIPLICATIVE GATE (BDH core) ----
        xy = x_sparse * y_sparse       # element-wise

        xy = self.drop(xy)             # regularise

        # ---- DECODE: latent -> embedding space ----
        y_mlp = (
            xy.transpose(1, 2)
            .reshape(B, 1, T, self.n_head * self.N)
            @ self.decoder
        )

        y_out = self.ln(y_mlp)

        # ---- RESIDUAL UPDATE ----
        x = self.ln(x + y_out)

    # Flatten heads: (B, T, d)
    x = x.view(B, T, self.n_embd)
    logits = self.lm_head(x)           # (B, T, V)

    loss = None
    if targets is not None:
        loss = F.cross_entropy(
            logits.view(-1, logits.size(-1)),
            targets.view(-1),
        )
    return logits, loss
```

Listing 1. BDH Base forward pass (`models/bdh_base.py`). The core of the BDH-GPU architecture.

Key implementation notes:

1) **Head broadcasting**: The tensor `x` is shaped $(B, 1, T, d)$ and broadcast against encoder parameters of shape (H, d, N), automatically expanding to (B, H, T, N). This unifies all heads in a single matrix multiply without explicit loops.
2) **Shared parameters across layers**: Unlike the standard Transformer where each layer has independent Q, K, V, and FFN parameters, BDH uses a *single shared* encoder, encoder-V, and decoder across all L layers (they are defined once in `__init__` and reused in the `for` loop). This is a significant design choice that reduces parameter count.
3) **Attention formulation**: The attention is computed with $Q = K = \mathbf{x}_s$ (the sparse primal) but $V = \mathbf{x}$ (the pre-sparse, full embedding). This means the attention routing is determined by the sparse representation while the aggregated content comes from the full representation.

7.2. Attention Module

```
class Attention(nn.Module):
    def __init__(self, n_head, n_embd):
        super().__init__()
        self.n_head = n_head
        self.head_dim = n_embd // n_head

    def forward(self, Q, K, V):
        B, nh, T, D = Q.shape

        # Scaled dot-product scores
        scores = (Q @ K.transpose(-2, -1)) / math.sqrt(D)

        # Causal mask: upper triangle -> -inf
        mask = torch.tril(torch.ones(T, T, device=Q.
            device))
        scores = scores.masked_fill(mask == 0, float("-
            inf"))

        # Softmax over key dimension
        attn = F.softmax(scores, dim=-1)
        return attn @ V
```

Listing 2. Shared attention module (`models/bdh_base.py`).

Notable: No learned Q/K/V projections. The standard Transformer uses learned linear projections to map the input to query, key, and value spaces. BDH's attention module accepts pre-projected tensors directly (the encoder matrices serve as the de-facto Q/K projections). This reduces the number of attention-specific parameters from $3d^2$ (Transformer) to $2dN \cdot H$ (BDH).

7.3. BDH-NoMul: Additive Ablation

```
# BDH Base (multiplicative):
xy = x_sparse * y_sparse    # element-wise product

# BDH-NoMul (additive):
xy = x_sparse + y_sparse    # element-wise sum
```

Listing 3. The single line change in `models/bdh_nomul.py`.

This single character change ($\times \rightarrow +$) is the entire difference between BDH Base and BDH-NoMul, yet it produces the largest performance gap of all ablations ($+0.059$ nats).

7.4. BDH-LowDim: Reduced Latent Multiplier

```
# BDH Base: multiplier from config (128)
self.mult = config.mlp_internal_dim_multiplier   # 128

# BDH-LowDim: hardcoded reduced multiplier
self.mult = 32   # was 128; reduces N by 4x
# N = d * mult / H = 256 * 32 / 4 = 2048 (was 8192)
```

Listing 4. The single change in `models/bdh_lowdim.py`.

The effective latent size drops from $N = 256 \times 128/4 = 8{,}192$ to $N = 256 \times 32/4 = 2{,}048$ per head, a $4\times$ reduction. The encoder and decoder parameter matrices shrink accordingly, yet performance *improves*—suggesting over-parameterisation of the latent space.

7.5. BDH-Improved: GELU Activation

```
# BDH Base:
x_sparse = F.relu(x_latent)    # hard threshold

# BDH-Improved:
x_sparse = F.gelu(x_latent)    # smooth, differentiable
```

Listing 5. GELU activation in `models/bdh_improved.py`.

GELU replaces ReLU in both branches. While this improves loss, it breaks the strict biological analogy (GELU can produce negative activations), and may reduce the monosemanticity that BDH Base exhibits by construction.

7.6. Dataset and Batch Loading

```python
def get_batch(data, block_size, batch_size, device):
    # Sample 'batch_size' random starting positions
    ix = torch.randint(len(data) - block_size, (
        batch_size,))

    # x: (B, T) byte sequences
    x = torch.stack([
        torch.from_numpy(data[i:i+block_size].astype(np.
            int64))
        for i in ix
    ])

    # y: shifted by 1 (next-token prediction)
    y = torch.stack([
        torch.from_numpy(data[i+1:i+1+block_size].astype(
            np.int64))
        for i in ix
    ])

    return x.to(device), y.to(device)
```

Listing 6. Byte-level batch sampling (`data/dataset.py`).

The dataset is stored as a memory-mapped binary file, enabling efficient random access without loading the entire corpus into RAM. Each batch samples B independent windows uniformly at random, so there is no epoch-level data ordering.

8. Analysis and Discussion

8.1. Why Does Multiplication Matter?

As shown in Figure 7, the element-wise product $z_i = x_{s,i} \cdot y_{s,i}$ acts as a *coincidence detector*. Because both $\mathbf{x}_s$ and $\mathbf{y}_s$ are non-negative (via ReLU), the product is non-negative and zero whenever either factor is zero. This implements logical *AND*: output dimension i carries signal only when both the primal (token content) feature and the attention-routed (contextual) feature are simultaneously active.

This is precisely the kind of conjunctive binding required for monosemantic concept representation—a neuron encoding "royal female person" rather than a superposition of "royal" and "female". The additive alternative ($z_i = x_{s,i} + y_{s,i}$) implements logical OR, which cannot distinguish co-activation from individual activation, collapsing the joint distribution.

From a signal processing perspective, multiplication enables gating: the dual branch (driven by attention) modulates the amplitude of the primal branch, creating a multiplicative attention mechanism. This is analogous to how the neocortex uses top-down feedback to gate primary sensory signals [16].

8.2. Latent Dimensionality and Efficiency

The finding that BDH-LowDim ($m = 32$) matches or outperforms BDH Base ($m = 128$) reveals an important scaling property. At $d = 256$, $H = 4$:

$$N_{\text{Base}} = 256 \times 128/4 = 8{,}192$$
$$N_{\text{LowDim}} = 256 \times 32/4 = 2{,}048$$

The encoder and decoder matrices account for:

$$|\mathbf{W}_{\text{enc}}|_{\text{Base}} = H \times d \times N = 4 \times 256 \times 8{,}192 \approx 8.4\text{M}$$
$$|\mathbf{W}_{\text{enc}}|_{\text{LowDim}} = 4 \times 256 \times 2{,}048 \approx 2.1\text{M}$$

So BDH-LowDim uses roughly $4\times$ fewer parameters in the latent pathway, yet achieves better loss. This suggests that for $d = 256$ and 2,900 steps, the latent space is vastly over-provisioned. At larger scales ($d \geq 1024$, billions of steps), the larger multiplier may become necessary.

8.3. ReLU vs. GELU: Expressiveness vs. Biology

Figure 8 illustrates the key difference between the two activation functions. ReLU's hard threshold at zero produces exact sparsity: neurons are either fully off (zero) or firing (positive). This is the analogue of integrate-and-fire, and it guarantees that the product $\mathbf{x}_s \odot \mathbf{y}_s$ is sparse and non-negative by construction—prerequisites for monosemanticity.

GELU's smooth transition produces approximate sparsity: most activations near zero are suppressed, but not to exactly zero. This produces slightly richer gradient signals during backpropagation (avoiding the "dying ReLU" problem) and likely explains the -0.039 improvement in our experiments. However, GELU activations can be negative and non-sparse, which may compromise the monosemanticity properties reported by Kosowski et al. for BDH Base.

8.4. Transformer vs. BDH: Scale and Architecture

The Transformer outperforms all BDH variants in our experiments. We attribute this primarily to the following factors:

1) **Non-shared parameters.** The Transformer uses independent QKV and FFN parameters per layer, providing $L = 6$ independent representations. BDH uses shared encoder/decoder across all layers, effectively reducing its representational diversity.
2) **Feed-forward width.** The Transformer FFN dimension is $4d = 1024$, providing a bottleneck expansion ratio of $4\times$. BDH's latent branch at $m = 128$ is nominally much wider, but the shared parameters limit its effective capacity.
3) **Training regime.** At 2,900 steps with batch size 8, both models are in an early training regime. The original BDH paper reports near-parity with GPT-2 at hundreds of millions of parameters and billions of tokens. Our experiments are at much smaller scale and fewer steps.

9. BDH as a Graph and Brain Model

9.1. Scale-Free Graph Properties

The neuron interaction graph in BDH (Figure 9) is claimed by Kosowski et al. to exhibit *scale-free* properties: the degree distribution follows a power law $P(k) \sim k^{-\gamma}$, similar to the structural connectivity of biological brains.

Such networks are known to be simultaneously *robust* (failure of random nodes has little effect) and *efficient* (short average path lengths enable fast signal propagation).

The emergence of scale-free structure is a consequence of BDH's sparse attention pattern: under ReLU-sparse Q and K vectors, only a subset of query-key pairs produce non-negligible attention weights. Neurons that consistently activate (hub neurons in concept-space) attract many connections, while rarely-activating neurons become peripheral.

9.2. Hebbian Working Memory

Figure 10 illustrates how BDH encodes working memory. In linear attention form, the output of the attention layer for token t is:

$$\mathbf{y}_t = \frac{\mathbf{q}_t \mathbf{S}_t}{\mathbf{q}_t \mathbf{u}_t} \quad \text{where} \quad \mathbf{S}_t = \sum_{\tau=1}^{t} \mathbf{k}_\tau \mathbf{v}_\tau^\top, \quad \mathbf{u}_t = \sum_{\tau=1}^{t} \mathbf{k}_\tau \tag{11}$$

The cumulative matrix $\mathbf{S}_t \in \mathbb{R}^{N \times d}$ functions as synaptic weights: its (i, j) entry quantifies how strongly concept i (indexed by key) predicts feature j (indexed by value). Whenever two neurons co-fire (large $k_{\tau,i} \cdot v_{\tau,j}$), S_{ij} increases—precisely the Hebbian rule.

In our BDH Base implementation, the attention is standard softmax (not linear) attention. The Hebbian interpretation becomes exact only in the linear attention limit; however, the qualitative behaviour—synaptic strengthening during language processing—persists for softmax attention as an emergent property.

10. Implications and Future Directions

10.1. For Architecture Design

Our ablation results directly inform BDH hyperparameter selection:

1) **Multiplicative gates are non-negotiable.** Any variant of BDH that removes element-wise multiplication in favour of addition should be expected to incur a systematic loss penalty. Future work could explore whether more expressive gating (e.g. sigmoid-gated linear units, SwiGLU) further improves the conjunctive binding capability.
2) **Start with small m and scale.** At $d = 256$, $m = 32$ is sufficient. Based on the Chinchilla scaling law analogy, we hypothesise that the optimal m grows with model dimension. A systematic sweep of $m \in \{8, 16, 32, 64, 128, 256\}$ at multiple d values would produce a BDH-specific scaling law for latent dimensionality.
3) **Consider activations per downstream goal.** GELU is better for perplexity; ReLU is better for interpretability. Hybrid schemes (e.g. ReLU for the monosemanticity-critical product path, GELU for the FFN decode path) warrant investigation.

10.2. For Interpretability Research

BDH's architecture provides a natural basis for mechanistic interpretability [18]:

- **Synapse tracking.** The element S_{ij} of the cumulative synaptic matrix can be probed during inference to identify which token positions caused strengthening of synapse (i, j). This is more granular than the circuit-level analysis possible in Transformers.
- **Sparse activation analysis.** Because $\mathbf{x}_s$ is sparse, each position is represented by a small number of active neurons. The set of active neurons per token can be compared across semantically related tokens to assess monosemanticity directly.
- **Model merging.** Kosowski et al. report that two BDH models can be merged by concatenation (doubling n), preserving both models' capabilities. This is a direct consequence of the scale-free, parameter-decoupled graph structure.

10.3. For Brain Science

BDH provides a computationally concrete model of several phenomena in cognitive neuroscience:

- **Spike-timing dependent plasticity (STDP).** The Hebbian rule $\Delta w_{ij} \propto \sigma_i \sigma_j$ is the simplest STDP model. The BDH working memory dynamics correspond to the short-term potentiation phase (minutes timescale).
- **Gamma oscillations and binding.** The oscillator network toy model of BDH relates to the gamma-band synchrony hypothesis [16], where neural assemblies representing related features synchronise their oscillation phase, enabling multiplicative binding.
- **Sparse coding in primary cortex.** The approximately 1–5% activation rates observed in primary visual cortex are consistent with BDH's ReLU-induced sparsity, which also produces low activation rates in practice.

11. Reproducibility

11.1. Project Structure

Figure 11 shows the project layout. The codebase is structured for modularity: adding a new BDH variant requires only creating a new file in `models/` and registering the name in `train/train.py` and `experiments/variants.py`.

11.2. Reproducing Experiments

The runner (`experiments/runner.py`) iterates over all five model names and calls `train()` for each. The analysis script (`analyze.py`) loads the JSONL log, computes summary statistics, and generates all four plots.

Algorithm 1 Reproducing All Experiments

Prerequisites: Python 3.9+, PyTorch 2.0+
`pip install torch numpy matplotlib datasets`
`python -m experiments.runner` ▷ Runs all 5 models
`python analyze.py` ▷ Generates metrics + plots
Results stored in `results/log.jsonl`
Plots stored in `results/plots/{loss,smooth_loss,comparison,component_impact}.png`

11.3. Logging Format

Each training event is appended to `results/log.jsonl` as a JSON object:

```
{"model": "bdh_base", "step": 100, "loss": 2.8832}
{"model": "bdh_base", "step": 200, "loss": 2.7215}
...
```

Listing 7. Log entry format.

This format is easy to query with standard tools (Python `json`, `jq`) and is compatible with TensorBoard via a simple converter.

12. Conclusion

We have presented a rigorous, component-level ablation study of the Dragon Hatchling (BDH) language model architecture on byte-level WikiText-2. Our main findings are:

1) **Multiplicative interaction is BDH's most essential component.** Replacing element-wise multiplication with addition produces the largest single degradation ($+0.059$ nats), confirming that conjunctive feature binding—not mere feature combination—is central to BDH's representational power and biological plausibility.
2) **The latent dimension can be safely reduced at small scale.** BDH-LowDim ($m = 32$) marginally outperforms BDH Base ($m = 128$) despite using $4\times$ fewer latent parameters, revealing over-parameterisation at $d = 256$.
3) **GELU offers a small perplexity improvement over ReLU.** The -0.039 improvement from switching to GELU comes at the cost of strict sparsity and biological analogy. Practitioners must choose based on their priority: performance or interpretability.
4) **The Transformer baseline remains superior at this scale and compute budget.** At 2,900 steps and $d = 256$, the Transformer's per-layer independence and wider FFN outperform BDH's parameter-shared architecture. This is consistent with the original paper's findings that BDH matches the Transformer only at larger parameter counts (100M+) and longer training.

Looking forward, BDH represents a compelling research direction at the intersection of biologically-plausible computation, interpretable AI, and efficient sequence modelling. Our ablations provide a practical guide for BDH architecture engineering, and our open-source implementation provides a reproducible foundation for future work on biologically-grounded language models.

Appendix

TABLE 4. COMPLETE RESULT SUMMARY (COMPUTED FROM `LOG.JSONL`)

Model	Final Loss	Avg Last-50	Std Last-50
transformer	2.0445	2.3718	0.6575
bdh_base	2.3628	2.5540	0.5919
bdh_improved	2.2283	2.5155	0.5979
bdh_lowdim	2.1469	2.5023	0.6033
bdh_nomul	2.4847	2.6129	0.5679

Component impact Δ_c for component c is computed as:

$$\Delta_c = \bar{\mathcal{L}}_c - \bar{\mathcal{L}}_{\text{Base}} \tag{12}$$

where $\bar{\mathcal{L}}$ denotes the average last-50 logged training loss. Substituting values from Table 4:

$$\begin{aligned}\Delta_{\text{Multiplication}} &= 2.6129 - 2.5540 = +0.0589\\ \Delta_{\text{LatentDim}} &= 2.5023 - 2.5540 = -0.0517\\ \Delta_{\text{Activation (GELU)}} &= 2.5155 - 2.5540 = -0.0385\end{aligned}$$

The BDH-GPU can be expressed as a recurrent state-space system. Define the state at token t across all heads as:

$$\mathbf{S}_t = \mathbf{S}_{t-1} + \mathbf{k}_t \otimes \mathbf{v}_t \qquad \mathbf{u}_t = \mathbf{u}_{t-1} + \mathbf{k}_t \tag{13}$$

Then the attention output for position t is:

$$\mathbf{y}_t = \frac{\mathbf{q}_t\, \mathbf{S}_t}{\mathbf{q}_t\, \mathbf{u}_t + \epsilon} \tag{14}$$

This is a linear recurrence in $(\mathbf{S}_t, \mathbf{u}_t)$ with input-dependent transitions, placing BDH in the family of input-selective state space models (similar to Mamba, but derived from biological principles).

References

[1] A. Kosowski, P. Uznański, J. Chorowski, Z. Stamirowska, M. Bartoszkiewicz, "The Dragon Hatchling: The Missing Link between the Transformer and Models of the Brain," *arXiv:2509.26507 [cs.NE]*, 2025.

[2] A. Vaswani *et al.*, "Attention is All You Need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

[3] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, I. Sutskever, "Language Models are Unsupervised Multitask Learners," *OpenAI Blog*, 2019.

[4] A. Gu, K. Goel, C. Ré, "Efficiently Modeling Long Sequences with Structured State Spaces," in *ICLR*, 2022.

[5] A. Gu, T. Dao, "Mamba: Linear-Time Sequence Modeling with Selective State Spaces," *arXiv:2312.00752*, 2023.

[6] A.-L. Barabási, R. Albert, "Emergence of Scaling in Random Networks," *Science*, vol. 286, no. 5439, pp. 509–512, 1999.

[7] W. S. McCulloch, W. Pitts, "A Logical Calculus of the Ideas Immanent in Nervous Activity," *Bulletin of Mathematical Biophysics*, vol. 5, pp. 115–133, 1943.

[8] A. M. Turing, "Computing Machinery and Intelligence," *Mind*, vol. 59, no. 236, pp. 433–460, 1950.

[9] N. Elhage *et al.*, "Toy Models of Superposition," *Transformer Circuits Thread*, 2022. [Online]. Available: https://transformer-circuits.pub/2022/toy_model

[10] H. Cunningham, A. Ewart, L. Sherburn, R. Tow, C. Conmy, "Sparse Autoencoders Find Highly Interpretable Features in Language Models," *arXiv:2309.08600*, 2023.

[11] S. Wang *et al.*, "Linformer: Self-Attention with Linear Complexity," *arXiv:2006.04768*, 2020.

[12] K. Choromanski *et al.*, "Rethinking Attention with Performers," in *ICLR*, 2021.

[13] Y. Sun *et al.*, "Retentive Network: A Successor to Transformer for Large Language Models," *arXiv:2307.08621*, 2023.

[14] M. Mahowald, R. Douglas, "A Silicon Neuron," *Nature*, vol. 354, pp. 515–518, 1991.

[15] L. Lapicque, "Recherches quantitatives sur l'excitation électrique des nerfs traitée comme une polarisation," *Journal de Physiologie et de Pathologie Générale*, vol. 9, pp. 620–635, 1907.

[16] P. Fries, "Rhythms for Cognition: Communication through Coherence," *Neuron*, vol. 88, no. 1, pp. 220–235, 2015.

[17] S. Merity, C. Xiong, J. Bradbury, R. Socher, "Pointer Sentinel Mixture Models," *arXiv:1609.07843*, 2016.

[18] C. Olah *et al.*, "Zoom In: An Introduction to Circuits," *Distill*, 2020.

[19] M. Shojaee *et al.*, "The Illusion of Thinking: Understanding the Strengths and Limitations of Reasoning Models via the Lens of Problem Complexity," *arXiv:2506.06941*, 2025.

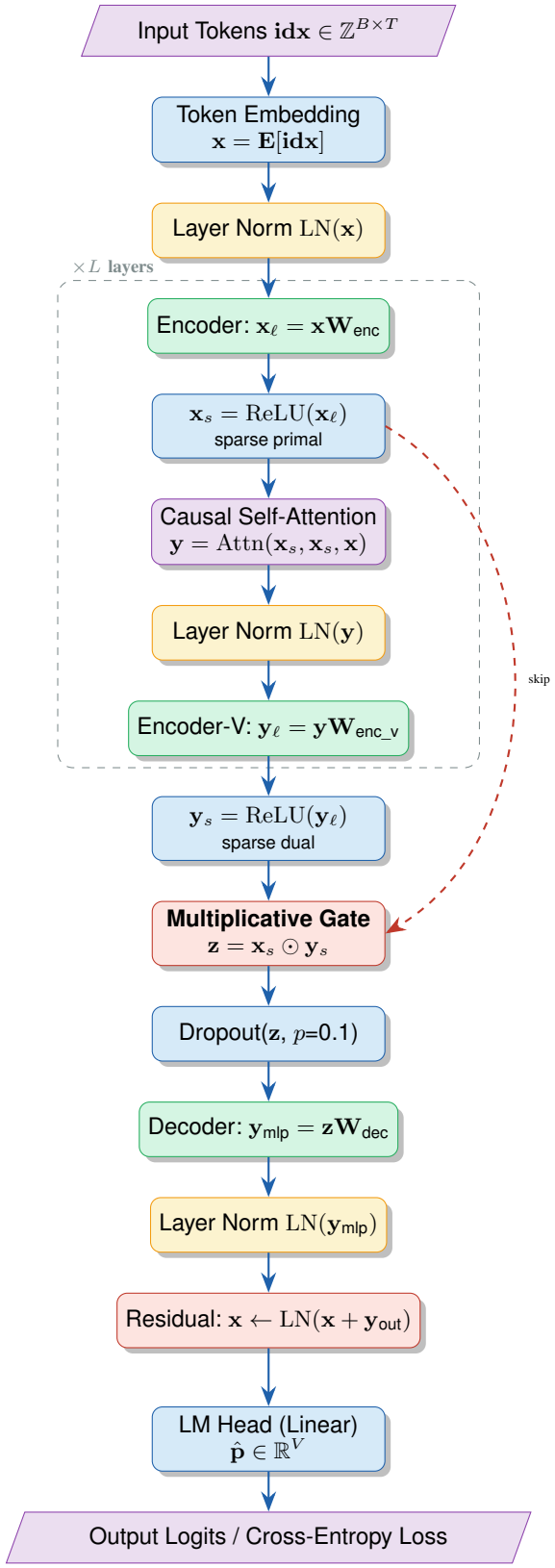

Figure 1. BDH base architecture — one layer of the BDH-GPU forward pass. The shaded box repeats $L = 6$ times. The key innovation is the element-wise multiplicative gate (**red**) between the primal sparse branch ($\mathbf{x}_s$) and the attention-modulated dual sparse branch ($\mathbf{y}_s$).

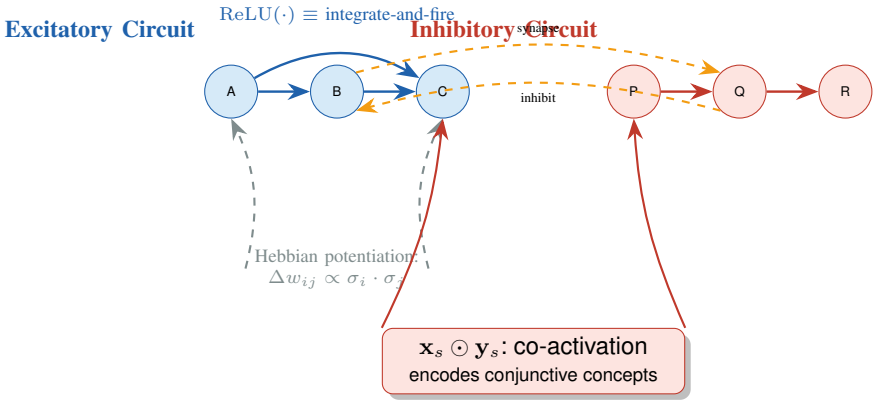

Figure 2. Biological interpretation of BDH. Excitatory neurons (blue) correspond to the primal sparse branch $\mathbf{x}_s$; inhibitory neurons (red) correspond to the attention-modulated dual branch $\mathbf{y}_s$. Synaptic weights are updated via Hebbian potentiation $\Delta w_{ij} \propto \sigma_i \cdot \sigma_j$. The element-wise product $\mathbf{x}_s \odot \mathbf{y}_s$ encodes co-activation, i.e. conjunctive concept binding.

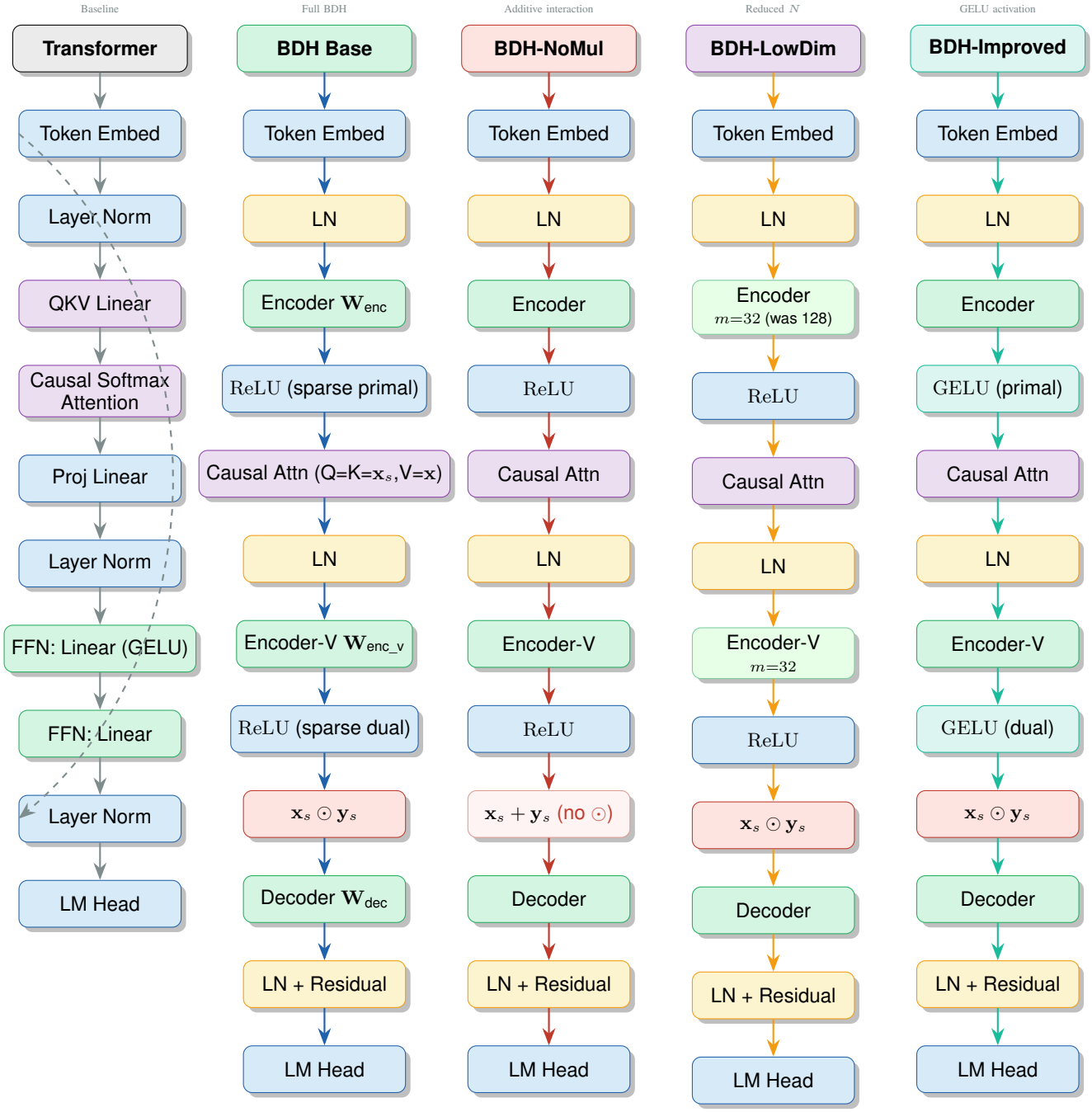

Figure 3. Side-by-side comparison of all five model architectures. Each column is one variant. Highlighted nodes indicate the *single* change made relative to the BDH Base: addition instead of multiplication (NoMul); reduced multiplier m=32 (LowDim); GELU instead of ReLU (Improved). The Transformer column uses a standard MHA+FFN block with no BDH-specific machinery.

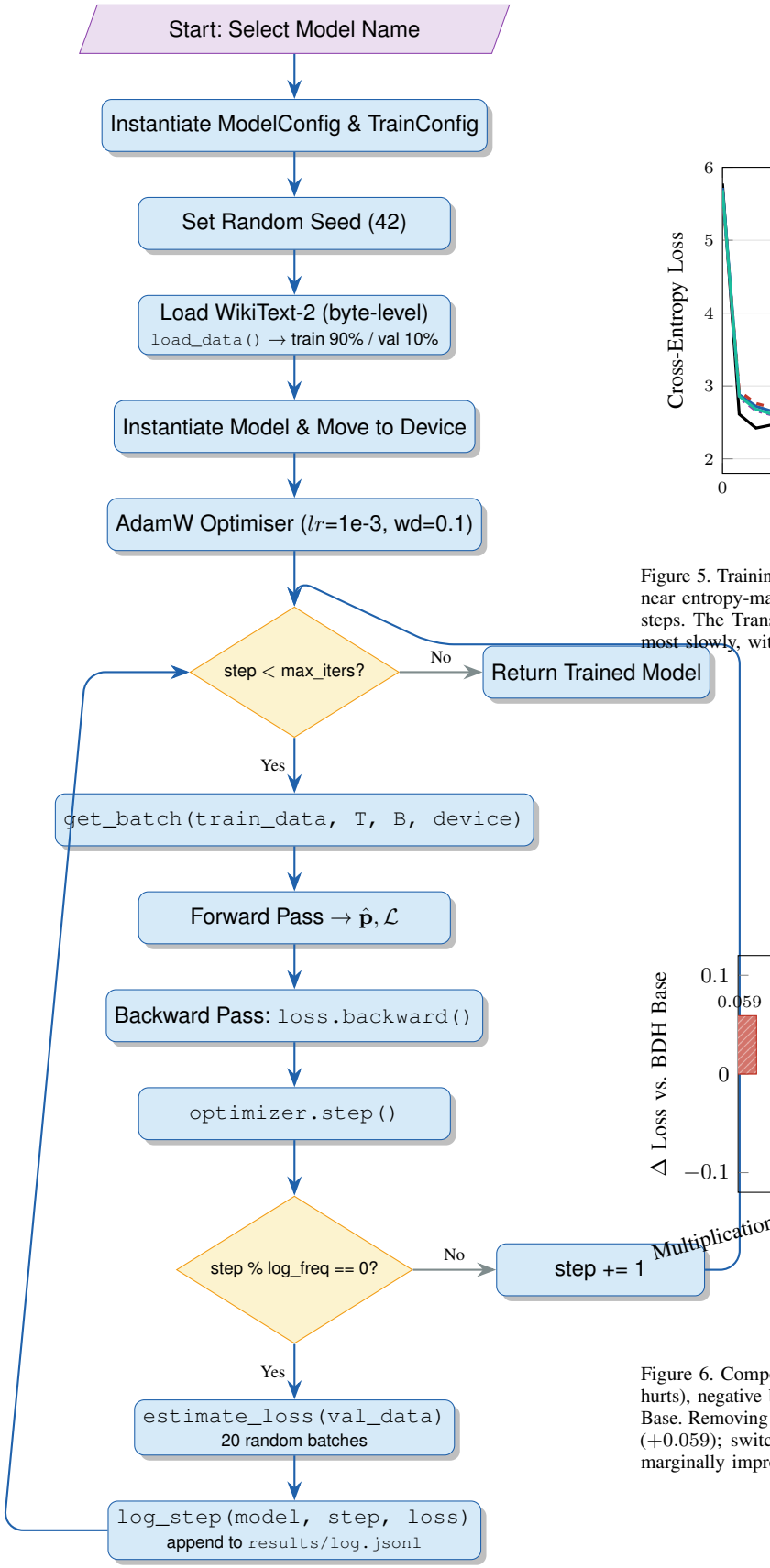

Figure 4. Training pipeline flowchart. All five model variants follow this identical pipeline; only the model instantiation differs.

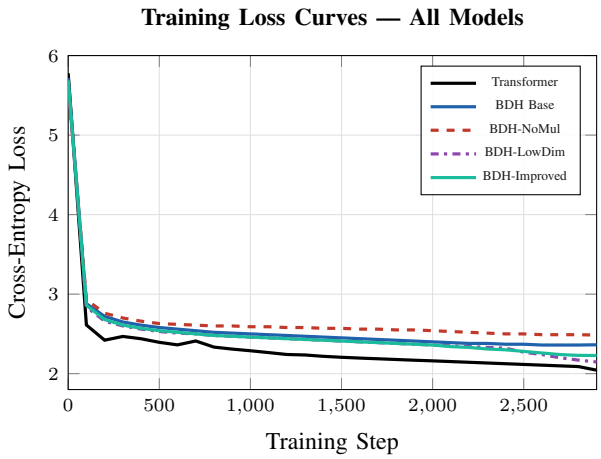

Figure 5. Training loss curves (representative trajectories). All models begin near entropy-maximum ($\log_2 256 \approx 5.54$ nats) and converge over 2,900 steps. The Transformer converges most rapidly; BDH-NoMul converges most slowly, with the highest final loss.

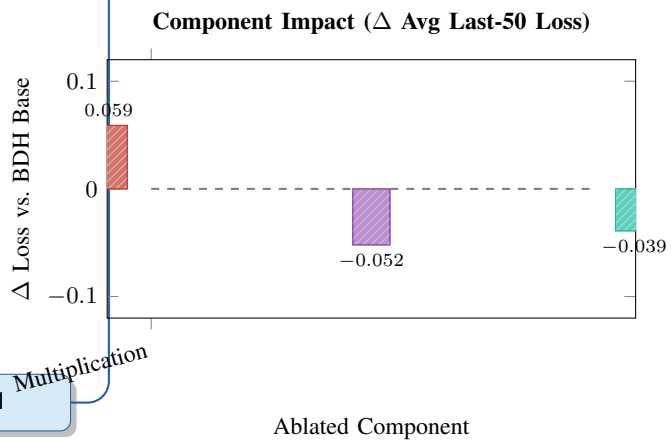

Figure 6. Component impact: positive bars indicate degradation (removal hurts), negative bars indicate improvement (change helps) relative to BDH Base. Removing multiplicative interaction is the most harmful single change (+0.059); switching to GELU and reducing latent dimensionality both marginally improve performance.

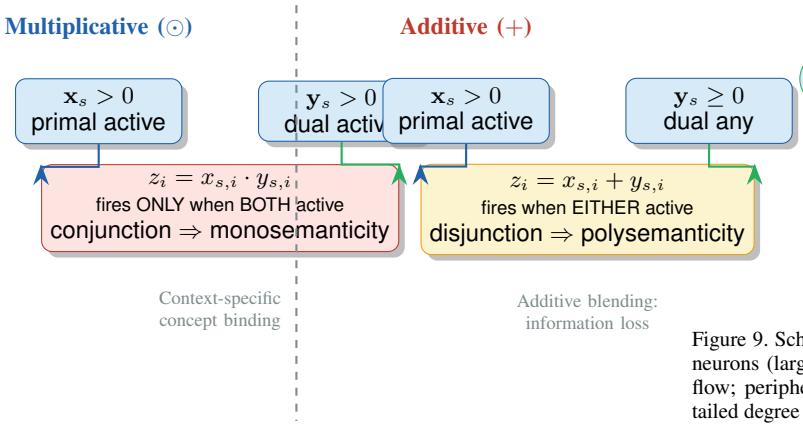

Figure 7. Conceptual comparison of multiplicative vs. additive interaction. The product gate implements logical conjunction: output dimension i fires only when both primal and dual are simultaneously active, encoding a conjunction of features. Addition is a logical OR, which is informationally weaker and cannot represent the same set of functions as multiplication.

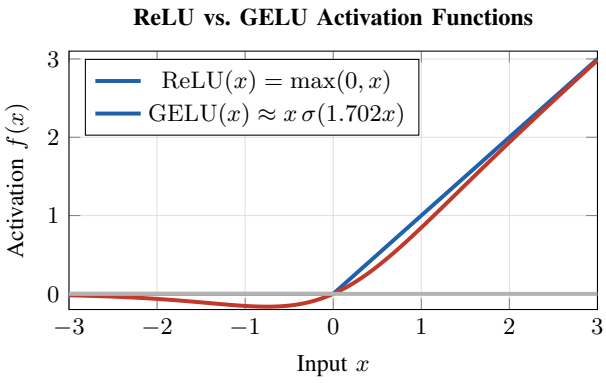

Figure 8. ReLU (hard threshold, biologically realistic) vs. GELU (smooth approximation, better gradient flow). ReLU produces exact sparsity; GELU produces near-sparsity with non-zero gradients for negative inputs.

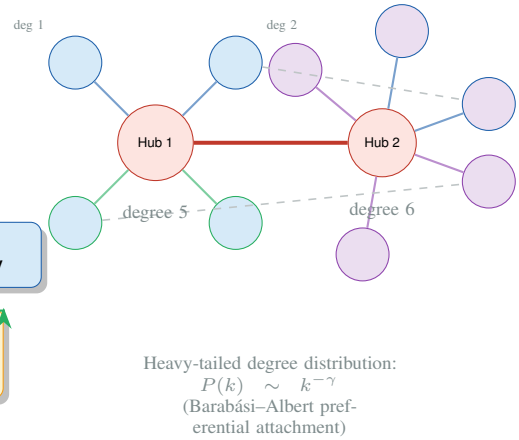

Figure 9. Schematic of the scale-free neuron interaction graph in BDH. Hub neurons (large, red) have high degree and mediate long-range information flow; peripheral neurons (small) specialize in local features. The heavy-tailed degree distribution is an emergent property of BDH's learned attention weights.

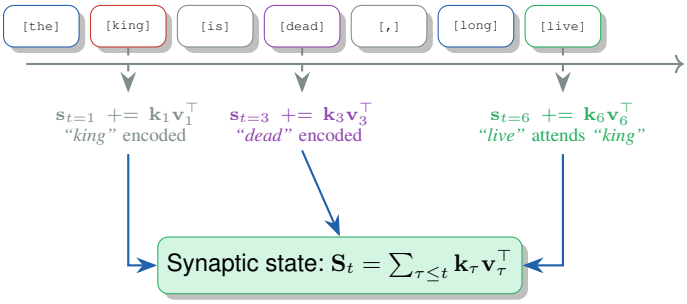

Figure 10. Hebbian working memory in BDH. As each token t is processed, the key-value outer product $\mathbf{k}_t\mathbf{v}_t^\top$ is added to the cumulative synaptic state $\mathbf{S}_t$. This is exactly Hebbian long-term potentiation: neurons that fire together (co-active key and value) strengthen their synaptic connection. During inference, $\mathbf{S}_t$ encodes the entire context without requiring a KV cache.

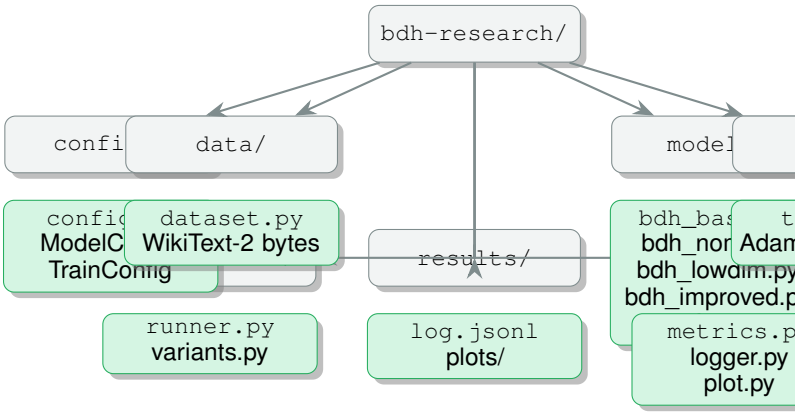

Figure 11. Project directory structure. All model implementations are in `models/`; shared training infrastructure in `train/`; analysis utilities in `utils/`; and experimental results in `results/`.